

面试复盘：R3 Router Replay 与 FlashInfer CUDA Graph 根因

日期：2026-08-18

用途：GPU Kernel / 算子岗真实施压面模拟后的复盘。两大主题——(A) R3 的机制与口径、(B) FlashInfer fused AllReduce+RMSNorm 的 rollout hang 根因。可背可查。

目标岗位：GPU Kernel / 算子。

0. 简历必改的两处（否则会被当场问穿）

1. R3 那条 "route mismatch 由 17%–19% 降至 0" —— 把两套不同口径缝在了一起，读过论文的面试官一问即穿。

- 改为："R2/R3 自然路由 mismatch 稳定在 17–19%（与论文同量级），R3 将 logprob drift 指标 f_tau_2 降低 36–145 $\times$ 、KL 降低 4–7 $\times$ "。只讲能自圆其说的。

2. FlashInfer 那条 —— 口述时主动界定贡献边界："定位到 FTZ 误判根因 + 在现用 FlashInfer 版本上落地位级比较修复 + 两卡最小复现与模型级 Graph on/off 回归通过"，不要让人误以为是自己发明的算法级修复。

主题 A: R3 Router Replay

A1. 一句话定义（记 id，不记分数）

推理时（vLLM/SGLang）记录每个 token、每层选中的 **top-k 专家 id**（`routed_experts`，形状 `[token, layer, top-k]`，只记 id，不记分数）；训练时（Megatron）跳过自己的 **top-k**、强制走这组记下来的专家，但用训练侧现算的 **router 分数 `s_train`** 做 gating softmax，然后正常前向、反向、更新。

换句话说："选哪些专家"用推理记录钉死；"给这些专家打几分、专家本身好不好"仍由训练侧学习。

为什么只记 id 不记分数（任选一条答都加分）：

1. **省传输/存储**：id 是 `[tok, layer, k]`，分数要 `[tok, layer, 专家总数]`，大一个数量级。
2. **保留训练侧梯度**：若直接用推理记录的分，router 被冻死、学不了；只固定"选谁"，分数用 `s_train` 现算，梯度能回到 router。
3. **精准打病根**：训推不一致的放大器是离散 top-k 换专家，不是分数的小数点漂移。

A2. RL 训练闭环（R3 落在哪）

===== 一个 RL step =====

① ROLLOUT (vLLM 推理引擎) — 逐 token 采样生成 (慢)

prompt —自回归采样→ response + rollout_log_probs (= π_{infer})

★R3 多记: routed_experts [token, layer, top-k] = I_{infer}

特点: FP8/fused kernel, 快, 无梯度, 无计算图

| (prompt, response, π_{infer} , I_{infer})

▼

② REWARD — 对 response 打分 (判题/RM) — 跟 R3 无关

|

▼

③ OLD_LOG_PROB 重算 (Megatron 训练引擎) ★纯前向, 无反向★ 整条并行前向 (快)

把①生成好的整条序列一把并行 forward → old_log_prob (= π_{train} , ★在计算图里★)

★R3 作用点1: 不自己 top-k, 回放 I_{infer} 那组专家

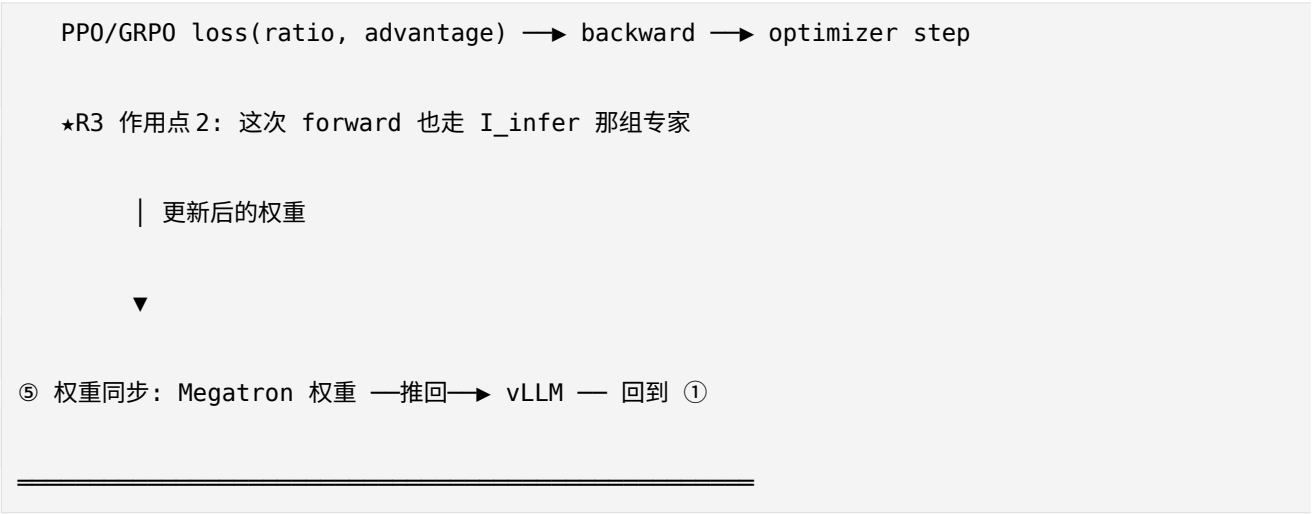
importance ratio $r = \exp(\text{old_log_prob} - \text{rollout_log_prob}) = \pi_{\text{train}} / \pi_{\text{infer}}$

【f_tau_2 / KL / k3_kl 就在这里量出来的】

| ratio + advantage(来自②)

▼

④ ACTOR UPDATE (Megatron) ★这里才有反向传播★



A3. 为什么要两次前向 (③ 不能省)

	① rollout 前向	③ old_log_prob 前向
引擎	vLLM	Megatron
方式	逐 token 采样 (慢)	整条并行 teacher-forcing (快)
产物	π_{infer} (一个数值)	π_{train} (计算图节点, 能 backward)
带梯度	✗	✓
精度/ kernel	FP8 / fused	BF16 / 训练 kernel
用途	提供经验数据	算 importance ratio, 供更新

- ③ 无法省掉: vLLM 那次前向不带梯度、精度也不同, 拿不去做训练更新; 只有 Megatron 里前向才能接着 backward。

- 2× 前向是 PPO/GRPO 的固有成本，不是 R3 引入的；R3 不增加前向次数，只让 ③ 那次已经要做的前向走对专家路径，额外开销 <3%（论文）。

- R3 在两个点起作用，第一个点没有反向：③ 纯前向 → drift 指标来源；④ 反向 → 训练稳定来源。不要说"R3 只在反向时有用"——会被反问"那 f_tau_2 哪来的"。

A4. R3 怎么起效果（因果链）

不用 R3：

① vLLM 生成时 router 选 [1,2] (I_infer)

③ Megatron 重算时自己 top-k 选成 [1,3] ← 换了专家！

→ 走完全不同的 FFN 子网络 → old_log_prob 与 rollout_log_prob 差很大

→ ratio 出现 0.1 或 10 极端值 → 梯度被放大 → 训练震荡/collapse

用 R3：

③ 强制用 [1,2] (回放 I_infer) → 走 rollout 真实那条路径

→ old_log_prob $\approx$ rollout_log_prob → ratio $\approx$ 1 → 梯度干净 → 稳

R3 降的是"后果"不是"路由本身"：路由不一致 (~18%) 客观存在、改不了 (rollout 已发生)；R3 让训练侧沿那条已发生的路径重算与更新，于是不一致带来的 **logprob drift** 被消除。所以成果指标是 drift，不是 mismatch=0。

A5. 既然"定死了", 为什么还要反向更新? (关键问题)

定死的和更新的不是同一样东西。

	是什么	R3 定死?	反向更新?
离散选择 <code>I</code>	选哪 top-k 个专家 (id)	☒ 定死 (用 <code>I_infer</code>)	☒ 本就不可导, 无所谓
gating 分数 <code>s_train</code>	router 给每个专家打的分	☒ 训练侧现算	☒ 更新
专家权重 <code>E_i</code>	被选中专家的 FFN 参数	☒	☒ 更新

R3 前向公式 (只看被选中的专家 `i`) :

```
g_replay,i = exp(s_train,i) / Σ_{j∈I_infer} exp(s_train,j)

y_replay    = Σ_{i∈I_infer} g_replay,i · E_i(x_train)
```

↑ `I_infer` 定死"选谁"

↑ 这两个量都是训练侧现算、可导

- `I_infer` 只是索引, 索引不可导 (dense MoE 的 top-k 本来也不可导) , 但它不需要梯度。
- 选定专家后, `g_replay` 用 `s_train` 现算 softmax → 梯度回到 **router 打分参数**。
- `E_i(x_train)` 是训练侧专家 FFN 真实前向 → 梯度回到 **专家权重**。

- **比喻**：定死专家选择 = "这道题必须交给张三李四做"；反向更新 = "他俩做得好不好、router 该多信任他们几分、他俩能力本身要不要提升"——全照常学。

replay bias：若 actor 更新多次、学习率高，当前 router 想换专家却被 mask 钉住，梯度会偏向旧路径。这是可接受代价（换来 ratio 可信、不崩），且 ⑤ 每 step 同步权重、重新 rollout，下一轮 I_infer 是更新后的 router 选的，路径**不会永久冻结，只在这一 step 内固定**。

A6. mismatch 的两套口径（简历冲突的根源）

	交付自一致口径	自然路由口径（正确）
开关	<code>metric_use_replay_target_as_actual=True</code>	<code>=False</code> ，或单独记训练侧自然 top-k
拿谁比谁	回放 target vs 实际 dispatch 的 target	训练侧 影子重算 的自然 <code>I_train</code> vs 推理记录 <code>I_infer</code>
R3 下的值	恒等于 0 （定义决定）	~18%，非零，R2/R3 都非零同量级
证明什么	只证明 <code>routed_experts</code> 被正确交付给 replay	证明训推路由本身有多不一致
偶发非零原因	采集/传输/对齐 bug（"丢记录"只在此成立）	本来就该非零

- 论文观测（自然口径）：router 级 ~10% 不一致、token 级 94% 至少一层不同、平均每 token ~6 层不同。实测 ~18% 同量级、方向一致。

- **bug 特征**：R2 $\neq$ 0 而 R3=0。

- **加分讲法**："我最初用 `=True`，R3 mismatch=0、topk_overlap=1，以为路由完全对齐。但发现 **R2 $\neq$ 0 而 R3=0** 的不对称——若 0 是对齐好的结果，R2 不该差这么多。顺着查下去才意识到那开关下是自证、恒为 0，只能当链路健康检查。于是改口径，R2/R3 回到 ~18% 同量级，R3 真正证据改用 logprob drift，并在交接文档里写成红线：谁再写'R3 mismatch 应为 0'就是错的。"

A7. 三句话钉死

1. R3 定死"选哪些专家"（离散、不可导、无需梯度）；反向更新"专家打分 $s_{\text{train}} + \text{专家权重 } E_i$ "（可导、照常学）——两者不是一回事。
2. R3 起效点在 ③（纯前向，drift 指标来源）和 ④（反向，训练稳定来源），本质是让训练前向复现 rollout 真实路径。
3. 降的是路由不一致的后果（logprob drift），不是路由不一致本身（~18%，改不了）。R3 只对 MoE 有用，对 dense 无同类收益。

主题 B：FlashInfer fused AllReduce+RMSNorm 的 rollout hang 根因

B1. 现象与定位路径（下游 → 根因）

条件：VLLM_COMPILE + FULL CUDA Graph + TP=2，H200，Qwen3-30B-A3B BF16。

```
rollout sample_tokens timeout + EngineDeadError
```

→ 两个 TP rank 在 sampler 之前就阻塞 (P1: 在 sampler 前 torch.cuda.synchronize() 就 hang)

→ GPU work 已 launch 但永不结束

→ 单变量隔离：只有关掉 fuse_allreduce_rms 这个 pass 才 PASS

根因矩阵（单变量控制）：

I D	单变量	结果	结论
R 0	compile + graph=NONE + TP=2	PASS 44.63s	compile 本身健康
R 1	compile + FULL + TP=2	HANG	最小复现
P 1	sampler 前 synchronize	HANG	replay 在 sampler 前未完成
P 4	blocking D2H copy	HANG	async copy 不是根因
R	关 async scheduling	HANG	output queue 不是根因

5

R	关普通 custom all-reduce	HANG	普通 all-reduce 不是根
6			因
F	关 fuse_allreduce_rms	PASS	单变量隔离到 fused
0		9.18s	pass
F	配置阶段自动 guard	PASS	最小修复首次通过
I		9.36s	
X			

所以 sampler / copy / output queue / MQ timeout 都是**下游等待点**，不是首因。

B2. 为什么必须 $TP \geq 2$ 才能复现（不是"内存塞不下"）

- $TP=1$ 没有跨卡规约 $\rightarrow$ fused allreduce+RMSNorm pass 不匹配/不生成 $\rightarrow$ 病根 kernel 压根不出现。
- 病根算子只存在于 $TP \geq 2$ 的通信路径上，所以最小复现的**最小规模就是 2 卡**。
- （易错点：不是"H200 单卡塞不下两份分片"——单卡 141GB 塞得下；是那条代码路径根本不走。）

B3. Lamport sentinel —— 是什么、为什么用

FlashInfer 的 fused one-shot allreduce（低延迟、Hopper 优化）用 **Lamport** 算法免除显式

barrier:

1. 每个 rank 先把接收 buffer 预置成"哨兵值"(sentinel) — 表示"数据还没到"
2. rank A 把数据跨卡 P2P 写进 rank B 的 buffer
3. rank B 不停轮询(polling)自己 buffer:
 - 还是哨兵值 → 对方没写到 → 继续自旋
 - 变成别的值 → 数据到了 → 读出来参与求和

核心：用"值变了没有"代替"barrier/信号来了没有" — 无 barrier 同步。

哨兵值 = **-0.0**，位模式 **0x80000000**（符号位=1，其余全 0）。选它因为：真实规约数据几乎不可能恰好等于它；数值上等于 0，不污染求和；有唯一位模式。

B4. FTZ bug（根因核心）

次正规数 (subnormal)：FP32 最小正规数约 $1.2e-38$ ；比它更小但非零（如 $1e-40$ ）的数是次正规数。

FTZ (Flush-To-Zero)：GPU 浮点模式，把次正规数直接冲刷成 0（为性能）。关键：**FTZ 把"负的次正规数"冲成 -0.0（保留符号位），不是 +0.0。**

原实现的错误：轮询用浮点比较识别哨兵（"是负数 → 还是哨兵 → 继续等；非负 → 数据到了"），假设"哨兵 -0.0 是负、真实数据非负"。

死循环：

1. 某位置的真实规约结果是合法的负小数，落进次正规区间，如 $-1e-40$
2. 对方 GPU 在 FTZ 下读它 → 冲刷成 -0.0 （和哨兵位模式一模一样）
3. 轮询："-0.0 是负数吗?" → 是 → "还是哨兵 → 数据没到 → 继续等"

↑ 但数据其实已经到了！
4. → 轮询永远退不出 → 自旋死循环
5. → 该 rank 的 fused allreduce kernel 永不结束

→ 永远发不出"all-reduce 完成" → 出不了 sampler

→ 另一个 rank / EngineCore 干等 → sample_tokens timeout → rollout hang

易错点：hang $\neq$ slow。观测到的**不是**"一个很长的 kernel"，而是"launch 了但没有 end 时间戳的 kernel"（GPU 停在某 kernel 上不往下走）。"算子耗时长导致超时"这个因果是错的。

B5. 修复：位级精确比较

原实现的错误本质：**用浮点语义（"是不是负数"）去判断一个本该按位模式判断的哨兵。** "是负数" $\supsetneq$

"是 -0.0"——负数有很多种，-0.0 只是其中一个特定位模式。

原来: <code>if (v < 0)</code>	→ 认为是哨兵	← 把所有负数当哨兵, 错
修复: <code>if (bit_pattern(v) == 0x80000000)</code>	→ 才是哨兵	← 只认唯一位模式

哨兵是一个位模式，不是一个"数值区间"。要按位比，不能按大小比。

贡献边界（面试主动说）：这是上游/社区已有的修法方向，我在我们这个 FlashInfer 版本上落地、
验证——定位到 FTZ 误判根因 + 位级比较落地 + 两卡最小复现 & 模型级 Graph on/off 回归全部
PASS，重复 Graph replay 未再出现 timeout/hang。

B6. 一段话口述版

TP $\geq$ 2 时会走 FlashInfer 的 fused one-shot allreduce+RMSNorm，它用 Lamport 算法免 barrier：接收 buffer 预置成 -0.0 哨兵，轮询到"值变了"就认为数据到了。原实现用浮点判断"是不是负数"来识别哨兵。但 H200 在 FTZ 模式下，会把合法的负次正规数冲刷成 -0.0——位模式和哨兵完全一样。于是当规约结果里出现一个负小数时，它被 flush 成 -0.0，轮询误判成"还是哨兵、数据没到"，自旋永远退不出，kernel 挂住，那个 rank 出不了 sampler，另一个 rank 和 EngineCore 干等到 timeout，表现成 rollout hang。修复是把哨兵判断从浮点"是否为负"改成位级精确比较 0x80000000——因为哨兵是一个唯一位模式，不是一个数值区间。定位靠两卡最小复现 + 单变量关掉 fuse_allreduce_rms 立刻 PASS + nsys 看到 kernel launch 了但不结束。

附：这次暴露、需要补的基础

- **TP vs DP**: TP 切**权重**（各卡算一部分，数据相同，靠 all-reduce 合并 → 所以才有那个 fused allreduce kernel）；DP 切**数据**（各卡一份不同数据、完整权重，梯度 all-reduce）。"一个卡一份数据"是 DP，不是 TP。
 - **hang ≠ slow**: nsys 里"kernel launch 了没有 end 时间戳" ≠ "一个很长的 kernel"，因果别讲反。
 - **RL 闭环两次前向各算什么**: 要练到能张口画出 ①→⑤。
-

待练（下次）

- Router GEMM 三级后端方案（DeepGEMM 主路径 → Triton full-K 低依赖 fallback → persistent 全覆盖 fallback；M>2048 回退 persistent）——kernel 岗必挖。
- OE 异步算子（Triton fused-hash、跨 step 状态、TP1/TP2/TP4 确定性通过率）。